

VisiChek

Breach Response Workflow

Backend Implementation Guide

Document	Backend API Reference for Breach Response
Audience	Engineering, Compliance, DPO, Security Officers
Version	1.0
Issue date	April 21, 2026
Companion to	VisiChek Breach Response Workflow (policy document)

Purpose

The policy document *VisiChek Breach Response Workflow* describes **what** VisiChek does when a suspected personal data breach is detected. This companion document describes **how** the VisiChek backend implements each of those steps — which HTTP endpoint the frontend calls for each action, what information it collects, who is authorised to call it, and what side effects (audit events, notifications, cache invalidations, background jobs) it triggers.

Every endpoint path below is mounted under the `/v1` API prefix. Most write operations (POST / PATCH / DELETE) return **HTTP 202 Accepted** with a `{ id, job_id, status: "queued" }` envelope — the work is executed by a Celery worker and clients poll GET `/v1/jobs/{job_id}` for completion. Read endpoints are wrapped in the standard success envelope `{ success, message, data, meta? }`.

REGULATORY CONTEXT

The Nigeria Data Protection Act 2023 and GAID 2025 require NDPC notification within 72 hours of becoming aware of a breach, and immediate notification to affected data subjects where the breach may pose high risk. The endpoints in this document are designed to meet those deadlines programmatically — every incident record stores a computed **notification_deadline** (`date_created + 72·3600` seconds) and a dedicated lookup endpoint surfaces incidents approaching that deadline.

1. Detection — Signals the Backend Exposes

The policy document lists detection channels such as failed login spikes, export activity anomalies, suspicious permission changes, and unusual API traffic. The VisiChek backend makes each of these signals queryable through the audit trail and admin action log, and enforces automatic containment (lockout) on the login path.

Failed login tracking (automatic)

The authentication layer records every failed login in the `login_attempts` MongoDB collection. After **MAX_FAILED_ATTEMPTS = 5** failed attempts for the same identifier (email), the account is locked for **LOCKOUT_DURATION_SECONDS = 900** (15 minutes). Both `authenticate_admin()` and `authenticate_system_user()` call `check_login_lockout(identifier)` before verifying credentials, so a locked account returns HTTP 429 regardless of whether the password is correct. Constants live in `security/password_policy.py`.

Audit trail queries

Two collections back audit reporting. The `audit_logs` collection is written by `log_action()` for admin/system actions (capturing *actor_id*, *action*, *target_entity*, *target_id*, *ip*, *device_signature*, *reason*, *occurred_at*). The `audit_trail` collection is written by `record_audit_event()` for domain events (capturing *actor_id*, *actor_role*, *action*, *resource_type*, *resource_id*, *tenant_id*, *details*, *request_id*, *timestamp*). The frontend surfaces both through a single filtered endpoint:

GET	/v1/audit-logs
Purpose	Investigator-facing feed of admin and system actions for a tenant.
Auth	super_admin, auditor, dpo
Collects	Query params: <i>actor_id</i> , <i>action</i> , <i>target_entity</i> , <i>date_from</i> , <i>date_to</i> , <i>start</i> (default 0), <i>stop</i> (default 100)
Returns	{ items: [...audit records with <i>actor_name_snapshot</i> , <i>ip</i> , <i>device_signature</i> , <i>occurred_at</i> ...], total }. First page without filters is served from a per-tenant precompute cache (60s TTL).

The audit query is the primary tool for the signals the policy lists: filter by `action="LOGIN_FAILED"` to find login spikes, filter by `target_entity="permission"` or `action="*.role_change"` to find suspicious permission changes, and date-bound the window to the suspected incident time.

Webhook event audit trail

Payment and infrastructure webhooks are persisted in the `webhook_events` collection for audit and idempotent replay. An application admin can enumerate them and re-process suspicious events:

GET	/v1/payments/webhooks/events
Purpose	Enumerate stored webhook events for investigation.
Auth	admin (application admin)
Collects	Query params: provider (stripe/flutterwave), processing_status, start, stop (max 200)
Returns	{ items: [...], meta: { total, start, stop } } including <i>provider</i> , <i>event_type</i> , <i>raw_payload</i> , <i>processing_status</i> , <i>created_at</i>

POST	/v1/payments/webhooks/replay/{event_id}
Purpose	Re-process a stored webhook event through the normal handler (recovery / integrity).
Auth	admin (application admin)
Returns	{ replayed_event_id, provider, result: { status: "processed" } }

2. First Response — Containment Endpoints

The policy calls for immediate containment: disabling compromised accounts, rotating credentials, revoking sessions, isolating affected systems. The backend exposes each of these actions as a dedicated endpoint, so the containment step can be executed from the operator console without direct database access.

Session revocation

Active sessions for any authenticated user are listed and revocable through the unified session endpoints. The session record carries device detection (*Chrome (Windows)*, *Safari (iOS)*...), IP address, and `last_active_at` so the incident owner can identify which session to kill.

GET	/v1/sessions
Purpose	List active sessions for the authenticated user (own sessions only).
Auth	Any authenticated role
Returns	Array of { id, device, location, ipAddress, deviceType, createdAt, lastActiveAt, isCurrent }
DELETE	/v1/sessions/{session_id}
Purpose	Revoke a single non-current session. Used when a specific device is suspected.
Auth	Any authenticated role
Returns	{ revoked: true }. Returns 400 if attempting to revoke current session.
POST	/v1/sessions/revoke-all
Purpose	'Log out of all devices' — revoke every session except the current one.
Auth	Any authenticated role
Returns	{ revoked_count: }

Credential rotation

Password rotation is enforced by the `password_change` path. It validates strength, checks the last five password hashes stored in `password_history`, and rejects common or breached passwords. On success, the gate cache for that user is invalidated so revoked permissions propagate immediately on the next request.

POST	/v1/auth/change-password
Purpose	Rotate password for any authenticated role with full policy enforcement.
Auth	Any authenticated role
Collects	{ current_password, new_password }
Returns	{ changed: true }. Returns 401 if current password wrong, 422 if new password fails policy (min 8 / max 128 chars, upper+lower+digit+special, no 4+ sequential or repeated chars, not in top-250 breach list, not reused in last 5).

Two-factor authentication enforcement

POST	/v1/auth/2fa/setup
Purpose	Generate TOTP secret and QR code (setup is pending until verified).
Auth	Any authenticated role
Returns	{ secret, otpauth_uri, qr_code_uri }. Returns 409 if 2FA already active.

POST	/v1/auth/2fa/verify
Purpose	Confirm pending TOTP setup and activate 2FA.
Auth	Any authenticated role
Collects	{ code: str } — 6-digit TOTP
Returns	{ enabled: true, backup_codes: [8 codes shown once only] }

DELETE	/v1/auth/2fa
Purpose	Disable 2FA. Requires password confirmation, not just a TOTP code.
Auth	Any authenticated role
Collects	{ password: str }
Returns	{ disabled: true }

POST	/v1/auth/2fa/backup-codes/regenerate
Purpose	Invalidate old backup codes and issue new ones (post-incident hygiene).
Auth	Any authenticated role
Collects	{ password: str }
Returns	{ backup_codes: [...] }

Account-level containment

For a tenant-wide compromise, application admins can offboard the entire tenant. This single call cancels the active subscription, deactivates every system user, marks the tenant inactive, records an audit event, and invalidates the plan cache — effectively locking the tenant out of the platform in one step.

POST	/v1/admins/tenants/{tenant_id}/offboard
Purpose	Atomic tenant deactivation — the 'kill switch' for a compromised tenant.
Auth	admin (application admin)
Collects	{ reason: str } — e.g., 'customer_request', 'non_payment', 'security_incident'
Returns	{ tenant_id, offboarded_at, reason, actions: { subscription_cancelled, users_deactivated_count, tenant_marked_inactive } }
Audit	Records audit event tenant.offboard

GET	/v1/admins/tenants/{tenant_id}/offboarding-summary
Purpose	Dry-run view of what offboarding would do (active users, subscriptions, open cases).
Auth	admin (application admin)
Returns	Summary object used by the frontend to render a confirmation dialog.

3. Opening and Tracking the Incident Record

The incident module is the backbone of the breach workflow. Every suspected or confirmed breach becomes a record in the `incident_logs` MongoDB collection. The record carries a computed **72-hour NDPC deadline** and exposes a dedicated endpoint for incidents approaching that deadline, so the incident owner never has to compute timers manually.

Incident taxonomy

Enum	Values
IncidentType	data_breach, unauthorized_access, data_export_exposure, device_loss, misconfiguration, third_party
IncidentStatus	open, investigating, contained, reported_to_ndpc, closed
risk_level	low, medium, high, critical (free-form string — guides data-subject notification decision)

Opening the incident record

POST	/v1/incidents
Purpose	Create a new incident record. Queues async write; pre-assigns the ID so the frontend can navigate to <code>/incidents/{id}</code> before the write completes.
Auth	super_admin, security_officer
Collects	incident_type (required enum), description (required), risk_level, data_affected, mitigation_steps, detection_time, ndpc_notified, ndpc_notified_at. Auto-populated from principal: tenant_id, reported_by. Auto-computed: notification_deadline = date_created + 72*3600.
Returns	HTTP 202 { id, job_id, status: "queued" }
Audit	record_audit_event(action="incident.create", resource_type="incident")

Tracking and updating the incident

GET	/v1/incidents
Purpose	List all incidents for the tenant, newest first.
Auth	super_admin, security_officer
Collects	Query params: start (default 0), stop (default 100)
Returns	Array of incident documents with total count. First page cached per-tenant (60s TTL).

GET	/v1/incidents/{incident_id}
Purpose	Retrieve full incident detail.
Auth	super_admin, security_officer
Returns	Full incident document or 404.

PATCH	/v1/incidents/{incident_id}
Purpose	Partial update — used throughout the lifecycle: add mitigation steps, mark NDPC notified, transition status, record resolved_at.
Auth	super_admin, security_officer
Collects	status, description, risk_level, data_affected, mitigation_steps, ndpc_notified, ndpc_notified_at, notification_sent_at, resolved_at. Auto-set: last_updated = now.
Returns	HTTP 202 { id, job_id, status: "queued" }
Audit	record_audit_event(action="incident.update")

Deadline tracking — the 72-hour regulator clock

GET	/v1/incidents/approaching-deadline
Purpose	Surface every incident where notification_deadline - now ≤ 24 hours AND notification_sent_at is null. This is what powers the 'incidents approaching deadline' dashboard widget and the daily ops digest.
Auth	super_admin, security_officer
Collects	Query params: start, stop
Returns	Filtered array of at-risk incidents.
Audit	For each returned incident, the caller may trigger notify_incident_deadline(user_id, user_type, incident_id, tenant_id) to push a warning into the recipient's notification feed.

HOW THE DEADLINE IS ENFORCED END-TO-END

(1) **POST /v1/incidents** stores notification_deadline = date_created + 259200. (2) Operations staff open the 'Approaching deadline' view (**GET /v1/incidents/approaching-deadline**) to see which incidents are inside the 24-hour warning window. (3) The system triggers notify_incident_deadline() which writes to notifications and fans out through the user's preferred channels (email / in-app). (4) When NDPC has been notified, the incident owner calls **PATCH /v1/incidents/{id}** with ndpc_notified: true, ndpc_notified_at: <ts>, notification_sent_at: <ts>, which removes the incident from the approaching-deadline feed.

4. Evidence Preservation

The policy requires preserving authentication logs, export logs, admin activity, database audit records, and infrastructure alerts without overwriting them. The VisiChek backend supports this by storing every significant action in append-only MongoDB collections that the incident owner can query and export.

Evidence category	Where it lives	How to retrieve it
Authentication logs	audit_logs + login_attempts	GET /v1/audit-logs?action=LOGIN_*
Admin & system activity	audit_logs	GET /v1/audit-logs (filter by actor_id / target_entity)
Domain event audit	audit_trail	Queried internally by get_audit_trail_for_resource() and exposed via compliance export.
Webhook / payment events	webhook_events	GET /v1/payments/webhooks/events
Session activity	sessions	GET /v1/sessions (with device, IP, last_active_at)
Background job audit	queue_job_log	GET /v1/jobs/{job_id} per job (task_id, actor, payload_redacted, status, result, error, timestamps)
Consent events	Consent log collection	GET /v1/compliance/consent-log
Deletion / anonymisation	Deletion log collection	GET /v1/compliance/deletion-logs
Full regulatory export	All of the above, zipped	GET /v1/compliance/export (streams ZIP)

Compliance export bundle

GET	/v1/compliance/export
Purpose	Stream a ZIP containing the tenant's data processing register, consent log, deletion log, retention policies, and sub-processor list. The file is what gets handed to regulators and auditors during a breach investigation.
Auth	super_admin, dpo, auditor
Returns	application/zip stream — bypasses the queue/cache layer so the bundle is always fresh at the moment of export.

Background job audit trail

Because every write is queued, the queue_job_log collection is itself a forensic artefact: it records task_id, task_key, actor, tenant_id, resource_id, redacted payload, status (queued → processing → succeeded / failed), result, error, and timestamps. The investigator can reconstruct the exact sequence of actions by

filtering by actor and time range.

GET	/v1/jobs/{job_id}
Purpose	Retrieve the queue_job_log entry for a specific job — used to track the result of any queued write (incident create/update, DSR, settings change, etc).
Auth	Owner of the job or an authorised admin
Returns	{ task_id, task_key, actor, tenant_id, resource_id, status, result, error, created_at, updated_at, payload_redacted }

5. Notifying Affected Parties

The policy distinguishes three notification targets: internal staff, affected customer organisations, and affected data subjects. The backend uses its in-product notification system for the first two and the public rights portal for the third.

Internal notifications — trigger helpers

Services never instantiate a notification directly; they call one of the trigger helpers in `services/notification_service.py`. All triggers are fire-and-forget, are persisted in the notifications collection, and respect the recipient's `notification_preferences` (in-app, email, digest, etc.).

Helper	When it fires
<code>notify_incident_deadline(user_id, user_type, incident_id, tenant_id)</code>	Warning that an incident's 72-hour NDPC deadline is within 24 hours. Links to <code>/app/incidents/{id}</code> .
<code>notify_dsr_submitted(user_id, user_type, dsr_id, tenant_id)</code>	New data subject request received — routes to the DPO. Links to <code>/app/dsr/{id}</code> .
<code>notify_visitor_check_in(host_user_id, user_type, visitor_name, tenant_id)</code>	Host receives info notice when their visitor checks in.
<code>notify_checkin_pending_approval(tenant_id, checkin_id, visitor_name, verified, purpose, host_employee_id=None)</code>	Broadcast to all receptionists when a walk-in needs approval.
<code>notify_appointment_reminder(user_id, user_type, appointment_id, visitor_name, tenant_id)</code>	30-minute reminder before a scheduled appointment.
<code>notify_subscription_alert(user_id, user_type, message, tenant_id=None)</code>	Billing/suspension warnings. Links to <code>/app/billing</code> .
<code>notify_new_user_added(admin_user_id, user_type, new_user_name, tenant_id)</code>	Success notice to admin after creating a new tenant user.

Notification feed endpoints (recipient-facing)

GET	<code>/v1/notifications</code>
Purpose	Paginated notification feed for the authenticated user.
Auth	Any authenticated role
Collects	Query params: <code>skip</code> , <code>limit</code> (default 20, max 100), <code>read</code> (bool)
Returns	Array of notification documents. First page cached per-user (30s TTL).

GET	/v1/notifications/unread-count
Purpose	Unread badge count for the topbar.
Auth	Any authenticated role
Returns	{ count: int }. Cached per-user 15s.

PATCH	/v1/notifications/{notification_id}/read
Purpose	Mark a single notification as read.
Auth	Any authenticated role
Returns	HTTP 202 { status: "queued" }

POST	/v1/notifications/read-all
Purpose	Mark all notifications as read for the current user.
Auth	Any authenticated role
Returns	HTTP 202 { status: "queued" }

GET	/v1/notifications/preferences
Purpose	Retrieve the user's notification channel preferences.
Auth	Any authenticated role
Returns	NotificationPreferencesOut

PUT	/v1/notifications/preferences
Purpose	Update channel preferences (email opt-out, digest frequency, etc.).
Auth	Any authenticated role
Collects	NotificationPreferencesUpdate
Returns	HTTP 202 { status: "queued" }

Notifying affected customers via support cases

When a breach affects a specific customer organisation, the application-admin support-case channel is the formal communication path. Each case carries priority and an SLA; the approaching-SLA view mirrors the incident approaching-deadline view.

GET	/v1/admins/support-cases
Purpose	List all tenant support cases (admin-side inbox).
Auth	admin (application admin)
Collects	Query params: status, priority, category (billing / technical / account / feature_request / data_privacy / other), tenant_id, assigned_admin_id, support_tier, start, stop
Returns	Array of cases. Page 1 with no filters cached 60s.

GET	/v1/admins/support-cases/approaching-sla
Purpose	Cases whose SLA deadline elapses within 24 hours — the 'at risk' queue.
Auth	admin (application admin)

POST	/v1/admins/support-cases/{case_id}/messages
Purpose	Reply to a case. Internal notes are never shown to the tenant.
Auth	admin (application admin)
Collects	{ body: str, is_internal_note?: bool }
Returns	HTTP 202 { status: "queued" }

POST	/v1/admins/support-cases/{case_id}/assign
Purpose	Assign a specific admin as case owner. Priority tenants get an email ping on assign.
Auth	admin (application admin)
Collects	{ admin_id: str }

POST	/v1/admins/support-cases/{case_id}/transition
Purpose	Move a case through its status workflow (open → acknowledged → in_progress → awaiting_tenant → resolved → closed, plus reopened).
Auth	admin (application admin)
Collects	{ status: SupportCaseStatus }

6. Data Subject Notification & Rights

GAID Article 7 requires that data subjects be notified immediately where a breach poses high risk to their privacy, rights, or freedoms. The VisiChek backend exposes two surfaces: an **admin-side DSR lifecycle** for the DPO to manage requests, and a **public rights portal** so a visitor can raise an access, correction, deletion, or consent-withdrawal request without having an account.

Public (unauthenticated) endpoints

POST	/v1/public/rights/request
Purpose	Visitor submits a data subject request. Collects identity handles and issues a verification token so the visitor can check status without authenticating.
Auth	Unauthenticated (public)
Collects	{ request_type: "access" "correction" "deletion" "consent_withdrawal", email?, phone?, tenant_id, details? }
Returns	{ request_id, status: "pending", verification_token, due_date, message }
Audit	Triggers notify_dsr_submitted() to alert the tenant's DPO.

GET	/v1/public/rights/request/{request_id}/status
Purpose	Visitor polls for request status with the verification token they received.
Auth	verification_token (query param, one-time)
Collects	Query param: verification_token
Returns	{ request_id, status, due_date }

POST	/v1/public/rights/withdraw-consent
Purpose	Visitor withdraws consent for future processing. Sets consent_withdrawal_at on all active visitor sessions for the handle.
Auth	Unauthenticated (public)
Collects	{ email?, phone?, tenant_id }
Returns	{ message, sessions_updated: int }

PATCH	/v1/public/rights/profiling-opt-out
Purpose	Visitor opts out of repeat-visitor profiling (ProfilingPreference goes from allowed to opted_out).
Auth	Unauthenticated (public)
Collects	{ email?, phone?, tenant_id }
Returns	{ message: "Profiling preference updated to opted_out" }

Admin DSR lifecycle

POST	/v1/dsr
Purpose	DPO creates a DSR on behalf of a visitor (e.g., received by phone).
Auth	super_admin, dpo
Collects	visitor_profile_id (required), request_type (DSRType enum), visit_session_id?, notes?. Auto-populated: tenant_id, admin_id, received_at, date_created. sla_deadline is computed by the service.
Returns	HTTP 202 { id, job_id, status: "queued" }
Audit	record_audit_event(action="dsr.create")

GET	/v1/dsr
Purpose	Paginated DSR list for the DPO dashboard.
Auth	super_admin, dpo
Collects	Query params: start, stop
Returns	Array of DSRs. Page 1 cached per-tenant 60s.

GET	/v1/dsr/{dsr_id}
Purpose	Full DSR detail.
Auth	super_admin, dpo

PATCH	/v1/dsr/{dsr_id}
Purpose	Update DSR — identity verification, status transition, notes, resolved_at.
Auth	super_admin, dpo
Collects	status (pending → in_progress → completed / rejected), identity_verified, notes, resolved_at. Auto: last_updated.
Returns	HTTP 202 { id, job_id, status: "queued" }
Audit	record_audit_event(action="dsr.update")

CONTROLLER / PROCESSOR SPLIT	
VisiChek acts as the processor; the customer organisation is the controller. The backend surfaces the facts (via the compliance export and audit trail), but the controller-side organisation leads visitor-facing communications unless the DPA specifies otherwise. The DSR admin endpoints let the DPO record the organisation's response inside VisiChek so the record stays consistent with any regulator submission.	

7. Compliance Register, Retention & Sub-Processors

These endpoints back the 'post-incident review' phase of the policy — updating the DPIA, revising retention policies, and tracking sub-processor obligations. They are scoped to a tenant and callable by the DPO or super_admin.

Data processing register

GET	/v1/compliance/register
Purpose	Retrieve DPR entries (processing activities, lawful basis, purpose).
Auth	super_admin, dpo, auditor
Returns	Array of DPR documents. Cached per-tenant 120s.

POST	/v1/compliance/register
Purpose	Record a new processing activity in the register.
Auth	super_admin, dpo
Collects	purpose, lawful_basis, ... (tenant_id auto)
Returns	HTTP 202 { id, job_id, status: "queued" }

GET	/v1/compliance/deletion-logs
Purpose	Audit of records deleted or anonymised by retention policies.
Auth	super_admin, dpo, auditor
Collects	Query params: start, stop
Returns	Array of deletion log entries. Cached per-tenant 120s.

GET	/v1/compliance/consent-log
Purpose	Consent given / withdrawn events in a time range.
Auth	super_admin, dpo, auditor
Collects	Query params: start_date, end_date, skip, limit (default 100, max 1000)

Retention policies

POST	/v1/retention-policies
Purpose	Define a retention rule (days, delete vs anonymise).
Auth	super_admin, dpo
Collects	scope (e.g., 'visitor_profiles'), retention_days, action ('delete' or 'anonymise')
Returns	HTTP 202 { id, job_id, status: "queued" }

GET	/v1/retention-policies
Purpose	List retention policies. Cached per-tenant 60s.
Auth	super_admin, dpo

PATCH	/v1/retention-policies/{policy_id}
Purpose	Modify a retention rule (e.g., tighten after a post-incident review).
Auth	super_admin, dpo
Returns	HTTP 202

Sub-processors

POST	/v1/sub-processors
Purpose	Register a sub-processor (AWS, third-party vendor, etc.).
Auth	super_admin, dpo
Collects	provider, purpose, jurisdiction, dpa_signed (bool), uses_data_for_training (bool)
Returns	HTTP 202

GET	/v1/sub-processors
Purpose	List all sub-processors for the tenant.
Auth	super_admin, dpo

PATCH	/v1/sub-processors/{sp_id}
Purpose	Update a sub-processor entry (e.g., flag DPA re-signed).
Auth	super_admin, dpo
Returns	HTTP 202

DELETE	/v1/sub-processors/{sp_id}
Purpose	Remove a sub-processor that is no longer in use.
Auth	super_admin, dpo
Returns	HTTP 202

8. End-to-End Scenarios

The two walkthroughs below show how the endpoints above chain together during a live incident. Timestamps are in the tenant's local time; backend storage is Unix seconds.

Scenario A — Compromised receptionist account

T+0	Security monitoring flags 42 failed logins for reception@tenant.example in 90 seconds. Because failed_count crossed MAX_FAILED_ATTEMPTS=5, the backend automatically locks the account for 15 minutes (login_attempts).
T+2m	Security officer pulls recent activity: GET /v1/audit-logs?actor_id=<reception_id>&date_from=<15m ago> to confirm whether any successful login occurred before lockout.
T+5m	Containment. Security officer calls POST /v1/sessions/revoke-all as the reception user (or via impersonation flow). Every device is logged out.
T+7m	Credential rotation. Receptionist is forced through POST /v1/auth/change-password and POST /v1/auth/2fa/setup → /verify on next sign-in.
T+10m	Incident record opens. POST /v1/incidents with incident_type="unauthorized_access", risk_level="medium", description of the spike, detection_time=<T+0>. Server returns { id, job_id, status: "queued" } and computes notification_deadline = T+10m + 72h.
T+30m	Evidence bundle. DPO exports the compliance archive via GET /v1/compliance/export and attaches the 24-hour slice of GET /v1/audit-logs to the incident record (PATCH /v1/incidents/{id} with updated mitigation_steps).
T+1h	Assessment. Investigation confirms no successful login; no personal data accessed. PATCH /v1/incidents/{id} sets status="contained" and risk_level="low". No NDPC notification required.
T+24h	Closure. PATCH /v1/incidents/{id} sets status="closed" and resolved_at=<now>. Post-incident review added to mitigation_steps.

Scenario B — High-risk visitor data exposure

T+0	Customer org reports that a subset of visitor records is visible to the wrong branch. Support case opened via POST /v1/support-cases (tenant side). Application admin sees it in GET /v1/admins/support-cases .
T+30m	Incident created. Security officer: POST /v1/incidents with incident_type="data_export_exposure", risk_level="high", description pointing to the misconfigured branch filter. 72-hour deadline now ticking.
T+1h	Containment. POST /v1/admins/tenants/{tenant_id}/offboard is <i>not</i> appropriate; instead, super_admin patches the branch configuration and the branding system confirms via GET /v1/branches that the filter is correct. Affected sessions are killed via POST /v1/sessions/revoke-all (per impacted user).

T+2h	Customer notification. Application admin uses POST /v1/admins/support-cases/{case_id}/messages with <code>is_internal_note=false</code> to post the initial customer notice (summary, data categories, containment steps).
T+4h	Data subject notification path opens. Affected visitors can use POST /v1/public/rights/request (<code>request_type=access</code>) with the verification token they receive. The public portal URL is linked from the customer's notice.
T+12h	Evidence. DPO pulls GET /v1/compliance/export ZIP and GET /v1/audit-logs?target_entity=visitor&date_from=... to scope exposure.
T+48h	NDPC notification. DPO calls PATCH /v1/incidents/{id} with <code>ndpc_notified=true</code> , <code>ndpc_notified_at=<ts></code> , <code>status="reported_to_ndpc"</code> . Incident disappears from GET /v1/incidents/approaching-deadline .
T+30d	Closure. PATCH /v1/incidents/{id} <code>status="closed"</code> , root cause + remediation captured in <code>mitigation_steps</code> . DPIA update via POST /v1/compliance/register ; retention rule reviewed via PATCH /v1/retention-policies/{id} .

9. Quick Reference — Endpoints by Role

Role	Endpoints they can call
admin (application admin)	<code>/v1/admins/*</code> , <code>/v1/admins/support-cases/*</code> , <code>/v1/admins/tenants/{id}/offboard</code> , <code>/v1/admins/tenants/{id}/offboarding-summary</code> , <code>/v1/payments/webhooks/events</code> , <code>/v1/payments/webhooks/replay/{id}</code> , <code>/v1/platform-settings</code>
super_admin (tenant)	<code>/v1/incidents/*</code> , <code>/v1/dsr/*</code> , <code>/v1/audit-logs</code> , <code>/v1/compliance/*</code> , <code>/v1/retention-policies/*</code> , <code>/v1/sub-processors/*</code> , <code>/v1/tenant-settings</code> , <code>/v1/branches/*</code> , <code>/v1/branding</code>
security_officer	<code>/v1/incidents</code> , <code>/v1/incidents/{id}</code> , <code>/v1/incidents/approaching-deadline</code>
dpo	<code>/v1/dsr/*</code> , <code>/v1/audit-logs</code> , <code>/v1/compliance/*</code> , <code>/v1/retention-policies/*</code> , <code>/v1/sub-processors/*</code>
auditor	<code>/v1/audit-logs</code> , <code>/v1/compliance/*</code> (read-only)
Any authenticated role	<code>/v1/sessions</code> , <code>/v1/auth/change-password</code> , <code>/v1/auth/2fa/*</code> , <code>/v1/notifications/*</code> , <code>/v1/user-settings</code> , <code>/v1/jobs/{job_id}</code> , <code>/v1/account</code> (delete)
Public (no auth)	<code>/v1/public/rights/request</code> , <code>/v1/public/rights/request/{id}/status</code> , <code>/v1/public/rights/withdraw-consent</code> , <code>/v1/public/rights/profiling-opt-out</code>

Key behavioural notes

bullet All write verbs return **HTTP 202** with `{ id, job_id, status: "queued" }`. Clients poll `GET /v1/jobs/{job_id}` for the authoritative result.

bullet All read responses are wrapped: `{ success, message, data, meta? }`.

bullet First-page reads (start=0, stop=100, no filters) are served from the precompute cache (60–120s TTL depending on resource).

bullet Gate cache (5-minute TTL) caches account-status + permission snapshots in Redis; state-changing writers explicitly call `invalidate_gate(user_id, role)`.

bullet Plan-enforcement middleware adds `X-Plan-Tier` and `X-Subscription-Status` headers to tenant responses.

bullet Queue job log (queue_job_log) is an immutable forensic artefact per task: task_id, actor, tenant_id, redacted payload, status, result, error, timestamps.

WORKING ASSUMPTION

If there is a credible possibility that personal data was exposed, altered, lost, exported without authorisation, or viewed by the wrong party — open the incident record and start the clock. The endpoints are cheap to call; the record can always be downgraded to status="closed" with risk_level="low" after investigation. Not opening a record is the only action that cannot be undone.